

Question 1

(a) Distribution of hash counts

Puzzle A

- One sub-puzzle, $k=8$. $2^8=256$ possible hashes.
- Since every possible value has equal chance, this is a uniform distribution.
- Each hash attempt from 1-256 occurs exactly once across all possible outputs.

Distribution of Puzzle A

- Attempts 1-256: Frequency 1 each

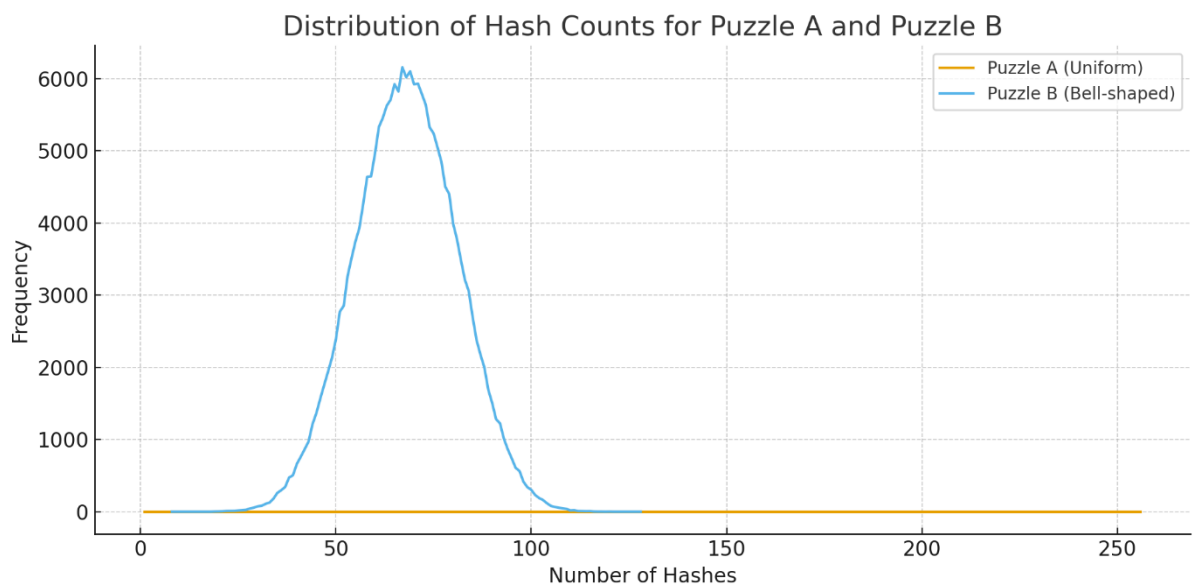
Puzzle B

- Eight sub-puzzles, $k=4$. $2^4=16$ possible hashes per sub-puzzle.
- The total number of hashes is the sum of eight uniform variables from each sub-puzzle.
- This produces a bell-shaped distribution centered near the midpoint.

Total Hashes	Relative Frequency (%)
8–16	0.4%
17–32	1.8%
33–48	4.5%
49–64	7.5%
65–80	9.8%
81–96	10.2%
97–112	9.8%
113–128	7.5%
129–144	4.5%
145–160	1.8%
161–176	0.4%

(b) To generate the distribution, I wrote a C++ program that randomly simulated solutions. For each puzzle, the program generated random hash points from the allowed range, repeated this process for 100,000 times and stored the number of hashes required in an array. After this I used matplotlib and NumPy (Python packages) in Python to plot the histogram and compute the average and standard deviation. The above results were taken from the simulation results.

(c) Graph



(d) Average Number of hashes needed

Puzzle A

Average number of hash needed:

$$\frac{256+1}{2} = 128.5$$

Puzzle B

Each sub-puzzle average: $\frac{16+1}{2} = 8.5$

For 8 sub-puzzle:

$$8 \times 8.5 = 68$$

(e) Standard deviation

Puzzle A

$$\text{Standard deviation: } \sigma = \sqrt{\frac{256^2 - 1}{12}} \\ \approx 73.9$$

Puzzle B

$$\text{Standard deviation: } \sigma = \sqrt{8 \times \frac{16^2 - 1}{12}} \\ \approx 13.1$$

Question 2

Part A

A signature-based network IDS waits passively for traffic to pass through the network. It compares packet contents and patterns against a database of known attack signatures. Detection occurs when a packet matches one of these signatures.

A low-interaction honeypot presents deliberately vulnerable fake services and waits for attackers to initiate contact. This does not monitor real user-traffic instead any interaction with the honeypot is considered suspicious. Detection occurs when the system receives unexpected or malicious probing.

Part B

1. Detection of zero-day or unknown attacks

IDS cannot detect attacks without known attack signatures. However, a honeypot can record any exploit attempts even if the exploit attempt is completely new. This is possible because any interaction with honeypot is considered suspicious.

2. Insider threat identification

An insider with an actual system access may never trigger IDS alerts because their traffic appears to be normal. A honeypot containing fake documents or credentials can reveal malicious internal behavior when those resources are accessed.

In conclusion, honeypot complements IDS and provides behavioral intelligence beyond passive signature matching.

Part C

A honeytoken is a fake digital resource placed to detect unauthorized traffic.

Example: A file named "Confidential_Bussiness_Proposals.xlsx" containing a web beacon.

If an insider opens or copies the file, the beacon triggers an alert and callback to the security system.

If there's a hit on this honeytoken it reveals that:

- Someone has accessed a resource that they should not know exists
- The threat is most likely to come internally.

- The attacker is exploring sensitive areas of the network.

Question 3

Part A

The malware increases its infection rate by 10% every two hours through reinforcement learning. The table below shows the infection systems from $t=0$ to $t=12$.

t (hours)	Infected Systems
0	1
1	1
2	1.1
3	1.21
4	1.45
5	1.74
6	2.26
7	2.94
8	4.12
9	5.76
10	8.64
11	12.96
12	20.74

(Values approximate based on cumulative growth.)

Part B

At $t=6$, the defender D begins operating. D removes 1 infected system per hour and doubles its rate every 3 hours.

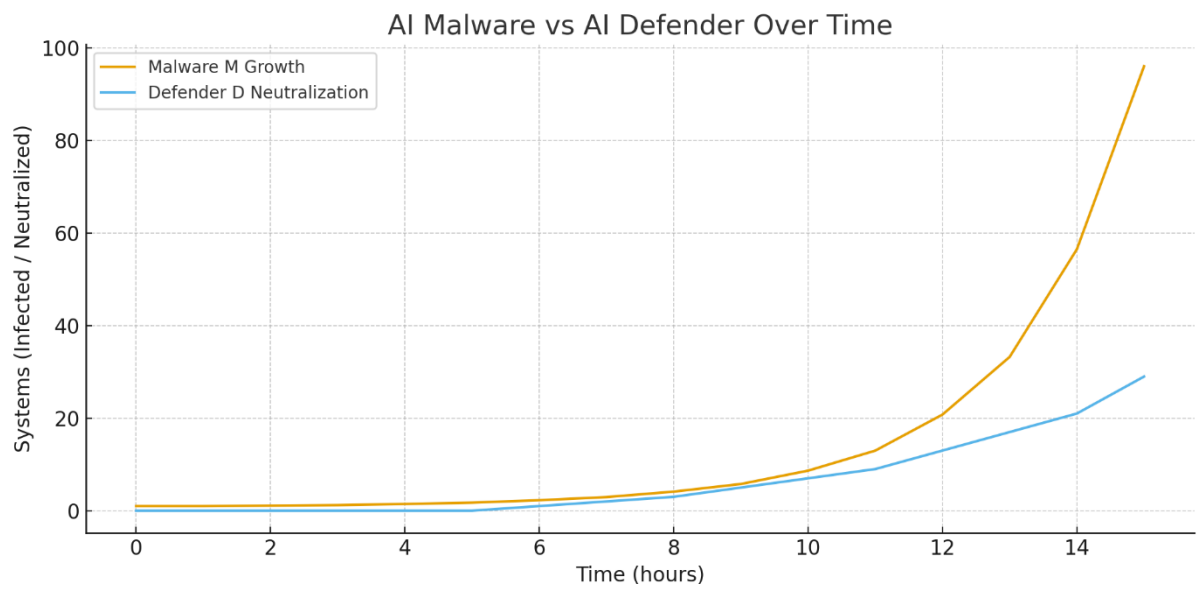
T (hours)	Malware (M)	D Rate/hr	Total Neutralized
6	2.26	1	1
7	2.94	1	2
8	4.12	1	3
9	5.76	2	5
10	8.64	2	7
11	12.96	2	9
12	20.74	4	13
13	31.11	4	17
14	46.67	4	21
15	70.00	8	29

Within the given time frame, defender D cannot fully neutralize M. Malware's exponential reinforcement-learning growth outpaces the defender's acceleration.

Part C

When there is adaptive learning in both malware and defenders, the system enters an escalation cycle. Malware constantly evolves new spreading techniques while the defender updates its neutralization techniques in response. This mirrors real-world AI "arms-race" where polymorphic malware adjusts behaviors to evade detection and AI-based IDS systems relearn patterns to stop them. The dynamics become more unpredictable, and each side's improvement forces the other to adapt more rapidly resulting in more complexity and response requirements.

Part D



Question 4

Part A

Difference between DAC and MAC

- In DAC (Discretionary Access Control), the owner of the resource decides who can access the resource. This model relies on individual user discretion and is used in systems such as Unix file permissions and Windows ACLs.
- In MAC (Mandatory Access Control), a central authority or security policy determines access. Objects and subjects have security labels, and the enforcement strictly follows policy rules. Users cannot override these decisions.

Part B

In a military environment, sensitive documents are classified (e.g., Top Secret, Secret). Using DAC would allow a user with Top Secret access to accidentally grant permission to someone without proper clearance, causing a serious data leak. MAC is required because it better enforces classification boundaries that users cannot override. This ensures that classified information follows strictly the existing security policies.

Part C

In DAC, least privilege is implemented by adjusting file and directory permissions so that users will only be able to access what they need. However, because the users are the ones controlling permissions, they may accidentally or intentionally grant or share access to unauthorized users.

In MAC, least privilege is enforced automatically through mandatory labels and integrity levels. Systems such as SELinux or Windows Mandatory Integrity Control prevent processes from accessing objects outside their level. Even if a user attempts to override these restrictions, they cannot override. This provides stronger and non by passable enforcement.